

Arrays:

To store a group of data together in a **sequential manner in computer's memory**, arrays can be one of the possible data structures. Arrays enable us to organize more than one element in consecutive memory locations; hence, it is also termed as structured or composite data type. The only restriction is that all the elements we wish to store must be of the same data type. It can be thought of as a box with multiple compartments, where each compartment is capable of holding one data item. Arrays support direct access to any of those data items just by specifying the name of the array and its index as the item's position (**sequence number as subscript**). Arrays are the most general and easy to use of all the data structures. An array as a data structure is defined as a set of **pairs (index, value)** such that with each index, a value is associated.

index—indicates the location of an element in an array **value**—indicates

the actual value of that data element

Here is an Example of C++ Array. How the Elements are accessed by the Indices.

```
1  #include <iostream>
2  using namespace std;
3  int main()
4  {
5      const int N{ 5 }; // the size of array should be constant
6
7      int arr[N] = {1,2,3,4,5}; // defining and initializing the Array
8
9      for (int i = 0; i < N; i++)
10     {
11         cout << arr[i] << endl;
12     }
13     return 0;
14 }
```

Fig.04 (Arrays)

Explanation:

In line number 7 a static array at stack has been defined and initialized with integer values. In line number 9 a for loop have been used to display contents of Array. Attention Here the subscript/index ([]) operator have been used. But you can also use pointer notation to access the elements of Array.



Fig. 05 (output)

Memory Representation of 1-D array:

The address of the i^{th} element is calculated by the following formula:

(Base address) + (Offset of i^{th} element from base address)

Here, base address is the address of the first element where array storage starts.

Address of $A[i] = \text{Base} + i \times \text{Size of element}$

All the elements of the array must be properly initialized before referring in any expression. It is important to note that arrays and their sizes are mostly defined statically, so it is not possible to change the size at the time of execution.

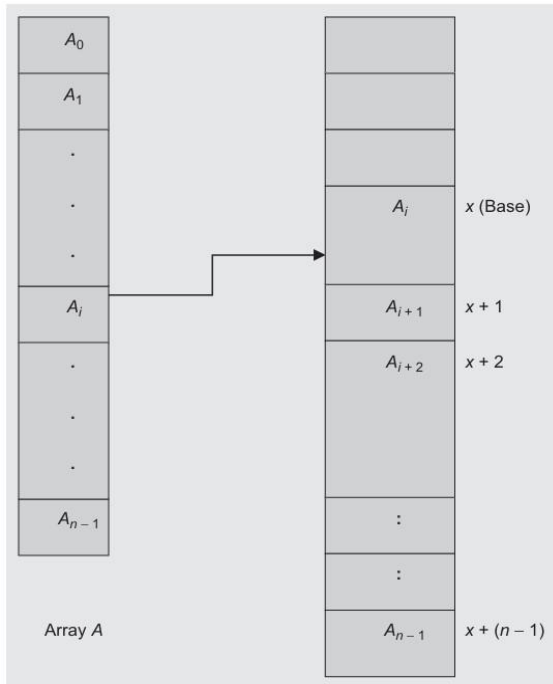


Fig.06 (1-D Array memory Address) **Memory Representation of Two-dimensional Arrays:**

Let us consider a two-dimensional array A of dimension $m \times n$. Though the array is multidimensional, it is usually stored in memory as a one-dimensional array. A multidimensional array is represented in memory as a sequence of $m \times n$ consecutive memory locations. The elements of a multidimensional array can be stored in the memory as

1. Row-major representation
2. Column-major representation

1. Row Major Representation:

In row-major representation the elements of Array are stored row-wise, that is, elements of the 0th row, 1st row, 2nd row, 3rd row, and so on till the $(m-1)^{\text{th}}$ row.

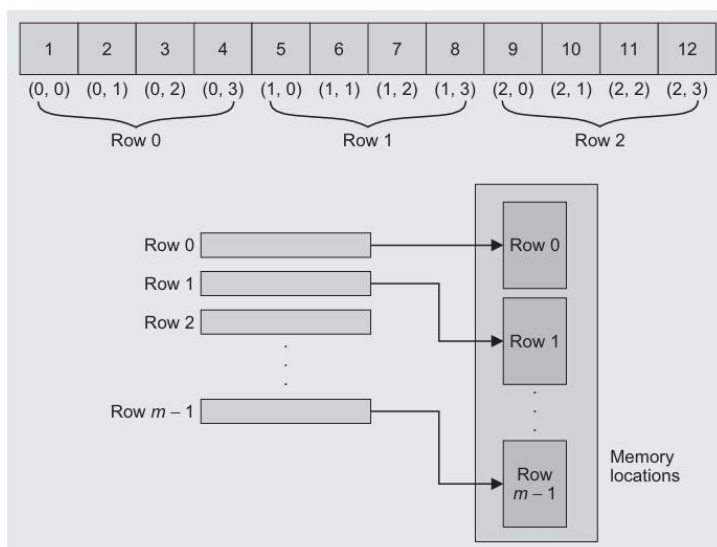


Fig.07 (row-major memory representation)

$$\begin{aligned}
\text{Address of } (A[i][j]) &= \text{Base Address} + \text{Offset} \\
&= \text{Base Address} + (\text{Number of rows placed before } i\text{th row} \times \text{Size of row}) \times (\text{Size of element}) + (\text{Number of elements placed before in } j\text{th element in } j\text{th column}) \times (\text{Size of element})
\end{aligned}$$

Here, size of a row is actually the number of columns n . The base is the address of $A[0][0]$.

$$\text{Address of } A[i][j] = \text{Base} + (i \times n \times \text{Size of element}) + (j \times \text{Size of element})$$

2. Column-major representation:

In column-major representation, $m \times n$ elements of a two-dimensional array A are stored as one single row of columns. The elements are stored in the memory as a sequence: first the elements of column 0, then the elements of column 1, and so on, till the elements of column $n - 1$.

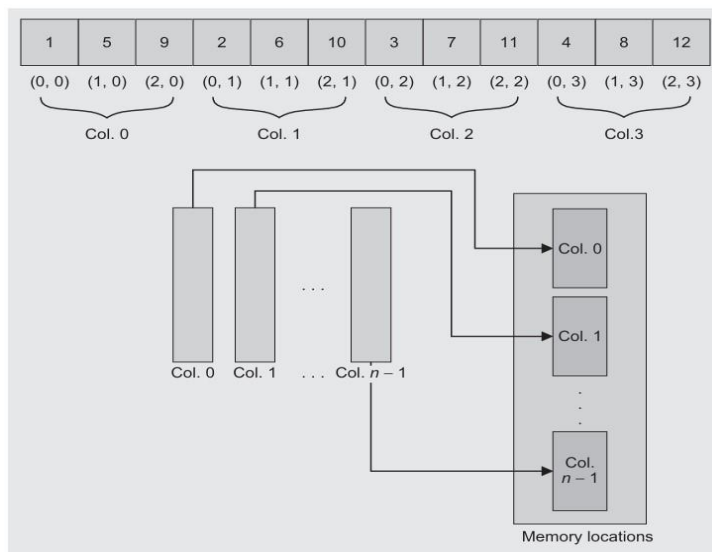


Fig.08 (column-major memory representation)

The address of $A[i][j]$ is computed as

$$\begin{aligned}
\text{Address of } (A[i][j]) &= \text{Base Address} + \text{Offset} \\
&= \text{Base Address} + (\text{Number of columns placed before } j\text{th column} \times \text{Size of column}) \times (\text{Size of element}) + (\text{Number of elements placed before in } i\text{th element in } i\text{th row}) \times (\text{Size of element})
\end{aligned}$$

Here, size of a columns is actually the number of rows m . The base is the address of $A[0][0]$.

$$\text{Address of } A[i][j] = \text{Base} + (j \times m \times \text{Size of element}) + (i \times \text{Size of element})$$

For size of the element = 1, the address is

$$\text{Address of } A[i][j] = \text{Base} + (j \times m) + i$$

N-Dimensional Arrays:

An n -dimensional $m_1 \times m_2 \times m_3 \dots \times m_n$ array A is a collection of $m_1 \times m_2 \times m_3 \dots \times m_n$ elements in which each element is specified by a list of n integers such as $k_1, k_2, k_3, \dots, k_n$ called subscripts where $0 \leq k_1 \leq m_1 - 1, 0 \leq k_2 \leq m_2 - 1, \dots, 0 \leq k_n \leq m_n - 1$. The elements of array A with subscripts $k_1, k_2, k_3, \dots, k_n$ is denoted by $A[k_1][k_2][k_3] \dots [k_n]$.

$$\text{Address of } A[k_1][k_2][k_3] \dots [k_n] = X + (k_1 \times m_2 \times m_3 \times \dots \times m_n) + (k_2 \times m_3 \times m_4 \times \dots \times m_n) + (k_3 \times m_4 \times m_5 \times \dots \times m_n) + (k_4 \times m_5 \times m_6 \times \dots \times m_n) + \dots + k_n$$

Memory address	Array elements	$m_1 = 2, m_2 = 3, m_3 = 4$
Base	$A[0][0][0]$	
Base + 1	$A[0][0][1]$	
Base + 2	$A[0][0][2]$	
Base + 3	$A[0][0][3]$	
Base + 4	$A[0][1][0]$	Base + $m_3 \times 1$
Base + 5	$A[0][1][1]$	
Base + 6	$A[0][1][2]$	
Base + 7	$A[0][1][3]$	
Base + 8	$A[0][2][0]$	Base + $m_3 \times 2$
Base + 9	$A[0][2][1]$	
Base + 10	$A[0][2][2]$	
Base + 11	$A[0][2][3]$	
Base + 12	$A[1][0][0]$	Base + $m_3 \times 3$
Base + 13	$A[1][0][1]$	
Base + 14	$A[1][0][2]$	
Base + 15	$A[1][0][3]$	
Base + 16	$A[1][1][0]$	Base + $m_3 \times 3 + m_2$
Base + 17	$A[1][1][1]$	
Base + 18	$A[1][1][2]$	
Base + 19	$A[1][1][3]$	
Base + 20	$A[1][2][0]$	
Base + 21	$A[1][2][1]$	
Base + 22	$A[1][2][2]$	
Base + 23	$A[1][2][3]$	

Fig.09 (3-D Array memory representation)

Sparse Matrices:

Sparse matrices are matrices in which a significant number of elements are zero or have some default value. These matrices are common in various scientific and engineering applications, such as finite element analysis, network analysis, and image processing, where the data is inherently sparse. Storing sparse matrices efficiently is essential to save memory and improve computational performance. In C++, there are several ways to represent sparse matrices:

1. Compressed Sparse Row (CSR) Format:

In the CSR format, you store three arrays:

- **data:** An array containing non-zero values.
- **indices:** An array containing the column indices of the corresponding values.
- **Index pointers:** An array that points to the beginning of each row in the **data** and **indices** arrays.

This format is efficient for matrix-vector multiplication operations and is widely used in numerical computing libraries like Intel Math Kernel Library (MKL) and SciPy.

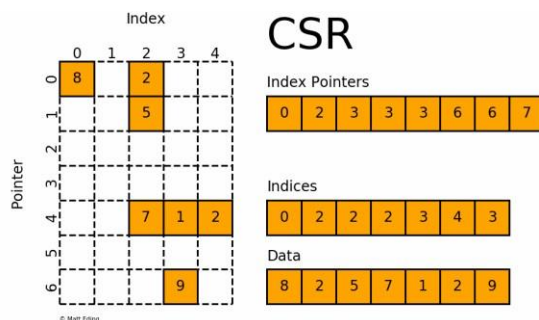


Fig.12 (Compressed Sparse Row)

2. Compressed Sparse Column (CSC) Format:

Similar to CSR, the CSC format stores non-zero values and column indices, but it uses a different indexing scheme. It is often more efficient for some operations like backslash (\) in MATLAB.

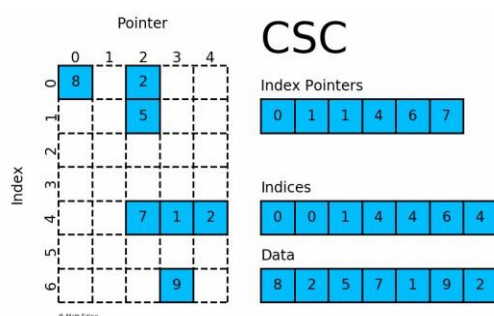


Fig.13 (Compressed Sparse Column)

3. List of Lists (LIL) Format:

In this format, you use a list of lists to represent the sparse matrix. Each list corresponds to a row and contains the non-zero elements along with their column indices. This format is straightforward but not very memory-efficient.

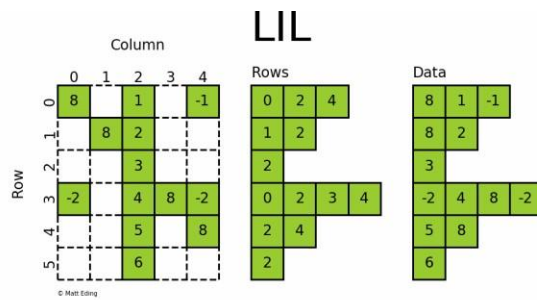


Fig.14 (List of List)

4. Coordinate List (COO) Format:

The COO format stores the (row, column, value) triplets of non-zero elements in a list. This format is simple but can be inefficient for certain operations.

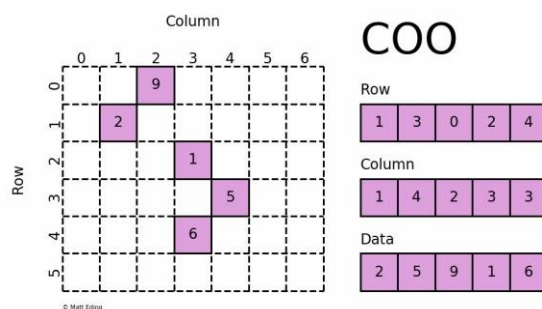


Fig.15 (Coordinate List)

5. Triangular Matrix Formats (CSR, CSC, etc.):

If your sparse matrix is triangular (upper or lower), you can use the CSR or CSC format and ignore the zero elements below or above the main diagonal. In a Lower triangular sparse matrix, all elements above the main diagonal have a zero value. This type of sparse matrix is also known as a lower triangular matrix. In a lower triangular matrix, $\text{Array}[i][j] = 0$ where $i < j$.

Similarly, in an upper triangular matrix, $\text{Array}[i][j] = 0$ where $i > j$. Similarly, a tri-diagonal matrix is also another type of a sparse matrix, where elements with a non-zero value appear only on the diagonal or immediately below or above the diagonal. In a tri-diagonal matrix, $\text{Array}[i][j] = 0$, where $|i - j| > 1$.

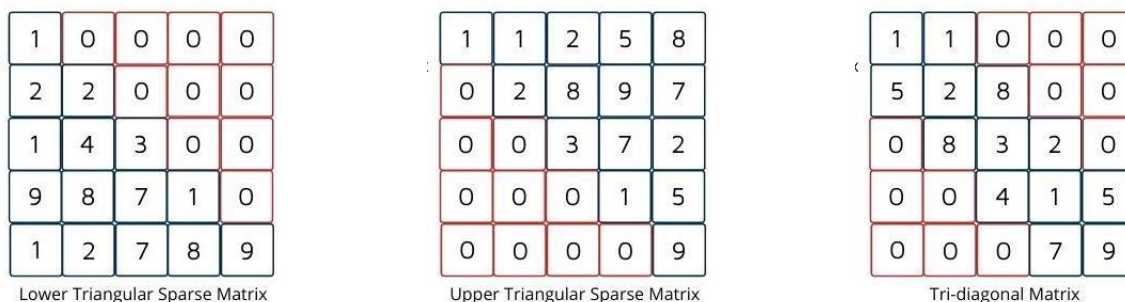


Fig.16 (Triangular Matrix)

6. Dictionary of Keys (DOK) Format:

DOK format uses a dictionary (associative container) to store (row, column) keys and their corresponding values. This format is useful for constructing a sparse matrix incrementally but may not be as efficient for matrix operations.

7. Eigen Sparse Matrix:

If you're using the Eigen library in C++, it provides a built-in SparseMatrix class that can efficiently represent and operate on sparse matrices. It supports various storage schemes, including compressed formats.

How to Select a Sparse Matrix Representation:

Choosing the right representation depends on your specific application and the operations you need to perform on the sparse matrix. The choice of format can significantly impact memory usage and computational performance. You may also need to consider libraries or frameworks like Eigen or Boost uBLAS that provide efficient implementations of sparse matrix operations.

In this Lab we will be using 2-D array-based representation of Sparse Matrix.

Each element of the matrix is uniquely characterized by its row and column positions. So a triple (i, j, value) can easily represent the non-zero elements of the matrix. In the sparse representation of a matrix, there are three columns. In the first row, we always specify the number of rows, columns, and non-zero elements (No_Of_NonZeroValues) in columns 1, 2, and 3, respectively. From the second row onwards, we store each non-zero element by its triple (i, j, value).

So in a sparse matrix, there are three columns and (No_Of_NonZeroValues + 1) rows. In general, for space reliability, $3 \times (\text{No_Of_NonZeroValues} + 1)$ should always be less than or equal to $m \times n$ where m = number of rows and n = number of columns. No_Of_NonZeroValues = Number of non-zero elements. In brief, for the alternate representation, we should have $3 \times (\text{No_Of_NonZeroValues} + 1) \leq m \times n$

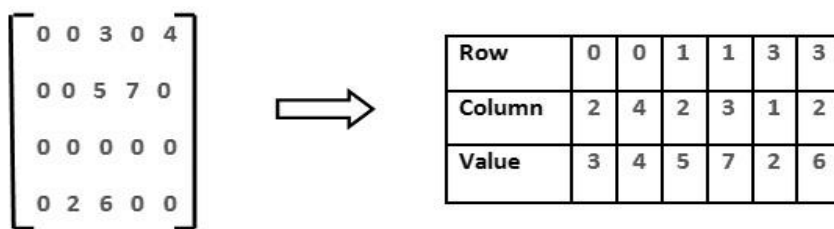


Fig.17 (Sparse Matrices) In left picture the Matrix is normal, in the right figure the matrix is sparse.

Task 01: Sparse Matrix

Create a 2D matrix class with well-defined data members and member functions to represent the matrix. You have been provided with a template for two classes, **which you can adjust as needed**. Consider how to track the count of non-zero elements in the matrix and contemplate the concept of composition. Should the Sparse Matrix class serve as a composite, with the Matrix class being composed within it?

```

1  ~
2  class Matrix
3  {
4  private:
5      int** data;
6      int rows;
7      int cols;
8
9  public:
10     Matrix(int numRows=0, int numCols=0);
11     ~Matrix();
12
13     void setValue(int row, int col, int value);
14     int getValue(int row, int col) const;
15     void display() const;
16 };
17
18

```

Fig.18 (Matrix)

Also, overload [][] operator for your matrix Class.

Define another class named SparseMatrix.

```
1 class SparseMatrix
2 {
3     private:
4         int non_zero;
5         const int cols { 3 };
6         int** S_Mat;
7     public:
8         SparseMatrix(int non_zero=0); // the constructor
9         void Read_SparseMatrix();
10
11         SparseMatrix AddSparseMatrix(SparseMatrix B);
12
13         void display() const;
14 };
```

Fig.19 (Sparse Matrix)

Instantiate two Matrix class objects and initialize them with random values obtained from a file. For instance, if you're creating a 10x10 matrix, it should consist of 100 elements. Out of these 100 elements, ensure that at least 85 percent of them are zeros.

The data of matrices should be read from a text file. Format of file should be like this for a 3x3 matrix.

```
0 3 0
0 0 0
1 0 0
```

Fig.20 (File Format)

Every line represents a row and number of elements in each line tell about elements that should be inserted in matrix in each row. This matrix will be formed from above file data.

```
0 3 0
0 0 0
1 0 0
```

You are tasked with populating two matrices using data from separate files. After successfully populating these matrices, convert them into valid Sparse Matrix objects (e.g., named sp_A, etc.). Once the data is read into these two matrices, each object of the Sparse Matrix class should now effectively represent a Sparse Matrix.

Following this, invoke a method within the Sparse Matrix class that performs the addition of these two Sparse Matrices and returns the resulting matrix. To conclude, display the initial two original matrices, the corresponding Sparse Matrices, and the resultant matrix obtained from the addition operation.